LABORATOR 3

Ana-Maria Corcodel

GRUPA 10 216

ACTIVITATEA 1

```
#include <omp.h>
#include <iostream>
#include <vector>
#include <random>
#include <algorithm>

using namespace std;

static const int N = 2000000;      // 2 milioane
static const int CUTOFF = 50000;   // prag pentru tasking

void merge_vec(vector<int>& a, int left, int mid, int right, vector<int>& temp) {
    int i = left, j = mid + 1, k = left;

    while (i <= mid && j <= right) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }

    while (i <= mid) temp[k++] = a[i++];
    while (j <= right) temp[k++] = a[j++];

    for (int p = left; p <= right; p++) {
        a[p] = temp[p];
    }
}

void mergesort_seq(vector<int>& a, int left, int right, vector<int>& temp) {
    if (left >= right) return;

    int mid = (left + right) / 2;
    mergesort_seq(a, left, mid, temp);
    mergesort_seq(a, mid + 1, right, temp);
    merge_vec(a, left, mid, right, temp);
```

```
}

void mergesort_task(vector<int>& a, int left, int right, vector<int>& temp) {
    if (left >= right) return;

    if (right - left + 1 <= CUTOFF) {
        mergesort_seq(a, left, right, temp);
        return;
    }

    int mid = (left + right) / 2;

    #pragma omp task shared(a, temp)
    mergesort_task(a, left, mid, temp);

    #pragma omp task shared(a, temp)
    mergesort_task(a, mid + 1, right, temp);

    #pragma omp taskwait
    merge_vec(a, left, mid, right, temp);
}

int main() {
    vector<int> a(N), temp(N);

    mt19937 rng(42);
    uniform_int_distribution<int> dist(1, 1000000);

    for (int i = 0; i < N; i++) {
        a[i] = dist(rng);
    }

    double start = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        mergesort_task(a, 0, N - 1, temp);
    }

    double end = omp_get_wtime();

    bool ok = is_sorted(a.begin(), a.end());

    cout << "Timp executie: " << (end - start) << " secunde\n";
    cout << "Sortare corecta: " << (ok ? "DA" : "NU") << "\n";
```

```
    return 0;
}
```

```
vboxuser@Ubuntu:~/Desktop$ nano activitatea1.c
vboxuser@Ubuntu:~/Desktop$ gcc -fopenmp activitatea1.c -o activitatea1
activitatea1.c:2:10: fatal error: iostream: No such file or directory
    2 | #include <iostream>
      |          ^~~~~~~~~~
compilation terminated.
vboxuser@Ubuntu:~/Desktop$ g++ activitatea1.c -O2 -fopenmp -o activitatea1
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=1
vboxuser@Ubuntu:~/Desktop$ ./activitatea1
Timp executie: 0.617254 secunde
Sortare corecta: DA
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=2
vboxuser@Ubuntu:~/Desktop$ ./activitatea1
Timp executie: 0.206142 secunde
Sortare corecta: DA
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=4
vboxuser@Ubuntu:~/Desktop$ ./activitatea1
Timp executie: 0.150459 secunde
Sortare corecta: DA
vboxuser@Ubuntu:~/Desktop$ S
```

Număr thread-uri	**Timp execuție (s)**	**Speedup**
1	0.617254	1.00
2	0.206142	2.99
4	0.150459	4.10

CONCLUZIE:

În această activitate am implementat Merge Sort folosind #pragma omp task, lansat corect dintr-o regiune parallel și un bloc single, conform modelului cerut în fișă. Introducerea unui prag cutoff a fost importantă pentru performanță: un prag prea mic generează foarte multe task-uri și crește overhead-ul de planificare, iar un prag prea mare reduce paralelismul disponibil. Varianta paralelă a obținut speedup pentru 2 și 4 thread-uri față de execuția cu 1 thread, însă accelerarea nu a fost perfect liniară din cauza costurilor de sincronizare și a operațiilor de merge.

ACTIVITATEA 2:

```
#include <omp.h>
#include <iostream>
#include <cmath>
#include <cstdlib>

using namespace std;

const int N = 20000000;      // 20 milioane elemente
const int CHUNK = 1000000;   // 1 milion per bloc

void init_block(double* a, int start, int end) {
    for (int i = start; i < end; i++) {
        a[i] = i * 0.5;
    }
}

void process_block(double* a, double* out, int start, int end) {
    for (int i = start; i < end; i++) {
        out[i] = sin(a[i]) + cos(a[i]);
    }
}

void consume_block(double* out, int start, int end, double &global_sum) {
    double local = 0.0;
    for (int i = start; i < end; i++) {
        local += out[i];
    }

    #pragma omp atomic
    global_sum += local;
}

int main() {
    double* a = new double[N];
    double* out = new double[N];

    int nb = N / CHUNK;
```

```
// token-uri pentru depend
int* tokens = new int[nb];
int* tokens2 = new int[nb];

double sumA = 0.0, sumB = 0.0;

// ==============================
// Varianta A - cu taskwait
// ==============================
double startA = omp_get_wtime();

#pragma omp parallel
{
   #pragma omp single
   {
      for (int b = 0; b < nb; b++) {
         int s = b * CHUNK;
         int e = s + CHUNK;

         #pragma omp task firstprivate(s, e)
         init_block(a, s, e);
      }
      #pragma omp taskwait

      for (int b = 0; b < nb; b++) {
         int s = b * CHUNK;
         int e = s + CHUNK;

         #pragma omp task firstprivate(s, e)
         process_block(a, out, s, e);
      }
      #pragma omp taskwait

      for (int b = 0; b < nb; b++) {
         int s = b * CHUNK;
         int e = s + CHUNK;

         #pragma omp task firstprivate(s, e)
         consume_block(out, s, e, sumA);
```

```
        }
        #pragma omp taskwait
    }
}

double endA = omp_get_wtime();

// ==============================
// Varianta B - cu depend
// ==============================
double startB = omp_get_wtime();

#pragma omp parallel
{
    #pragma omp single
    {
        for (int b = 0; b < nb; b++) {
            int s = b * CHUNK;
            int e = s + CHUNK;

            #pragma omp task firstprivate(b, s, e) depend(out: tokens[b])
            init_block(a, s, e);

            #pragma omp task firstprivate(b, s, e) depend(in: tokens[b]) depend(out: tokens2[b])
            process_block(a, out, s, e);

            #pragma omp task firstprivate(b, s, e) depend(in: tokens2[b])
            consume_block(out, s, e, sumB);
        }
    }
}

double endB = omp_get_wtime();

cout << "Varianta A (taskwait) timp: " << (endA - startA) << " secunde" << endl;
cout << "Varianta A suma: " << sumA << endl;

cout << "Varianta B (depend) timp: " << (endB - startB) << " secunde" << endl;
cout << "Varianta B suma: " << sumB << endl;
```

```
    delete[] a;
    delete[] out;
    delete[] tokens;
    delete[] tokens2;

    return 0;
}
```

```
vboxuser@Ubuntu:~/Desktop$ nano activitatea2.c
vboxuser@Ubuntu:~/Desktop$ g++ activitatea2.c -O2 -fopenmp -o activitatea2
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=4
vboxuser@Ubuntu:~/Desktop$ ./activitatea2
Varianta A (taskwait) timp: 0.37016 secunde
Varianta A suma: 5.3016
Varianta B (depend) timp: 0.162149 secunde
Varianta B suma: 5.3016
vboxuser@Ubuntu:~/Desktop$
```

CONCLUZIE:

Varianta cu depend (0.162 s) este mai rapidă decât cea cu taskwait (0.370 s), deoarece permite execuția în paralel a etapelor fără sincronizări globale inutile. Astfel, dependențele oferă o utilizare mai eficientă a thread-urilor și performanță mai bună.

ACTIVITATEA 3:

```
#include <omp.h>
#include <iostream>
#include <vector>

using namespace std;

static const long long N = 100000000; // 100 milioane

int main() {
    vector<int> v(N, 1);

    // 1. critical
    long long sum_critical = 0;
    double start1 = omp_get_wtime();

    #pragma omp parallel for
    for (long long i = 0; i < N; i++) {
        #pragma omp critical
        sum_critical += v[i];
    }

    double end1 = omp_get_wtime();

    // 2. atomic
    long long sum_atomic = 0;
    double start2 = omp_get_wtime();

    #pragma omp parallel for
    for (long long i = 0; i < N; i++) {
        #pragma omp atomic
        sum_atomic += v[i];
    }

    double end2 = omp_get_wtime();

    // 3. reduction
    long long sum_reduction = 0;
    double start3 = omp_get_wtime();

    #pragma omp parallel for reduction(+:sum_reduction)
    for (long long i = 0; i < N; i++) {
        sum_reduction += v[i];
```

```
    }

    double end3 = omp_get_wtime();

    cout << "critical:  " << (end1 - start1) << " sec, suma = " << sum_critical << "\n";
    cout << "atomic:    " << (end2 - start2) << " sec, suma = " << sum_atomic << "\n";
    cout << "reduction: " << (end3 - start3) << " sec, suma = " << sum_reduction << "\n";

    return 0;
}
```

```
vboxuser@Ubuntu:~/Desktop$ nano activitatea3.c
vboxuser@Ubuntu:~/Desktop$ g++ activitatea3.c -O2 -fopenmp -o activitatea3
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=1
vboxuser@Ubuntu:~/Desktop$ ./activitatea3
critical:  1.54317 sec, suma = 100000000
atomic:    0.759619 sec, suma = 100000000
reduction: 0.0910906 sec, suma = 100000000
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=4
vboxuser@Ubuntu:~/Desktop$ ./activitatea3
critical:  12.7036 sec, suma = 100000000
atomic:    1.45123 sec, suma = 100000000
reduction: 0.0212978 sec, suma = 100000000
vboxuser@Ubuntu:~/Desktop$
```

Metodă	Timp 1 thread (s)	Timp 4 thread-uri (s)	Observație
critical	1.54317	12.7036	Foarte lent, nu scalează
atomic	0.759619	1.45123	Mai bun, dar limitat
reduction	0.0910906	0.0212978	Cel mai rapid

CONCLUZIE:

reduction este cea mai eficientă metodă și scalează cel mai bine, deoarece evită sincronizarea frecventă. atomic este mai lent, iar critical este cel mai slab, deoarece introduce blocări puternice între thread-uri.

ACTIVITATEA 4:

```
#include <omp.h>
#include <iostream>
#include <vector>

using namespace std;

static const long long ITERS = 200000000;

struct alignas(64) PaddedCounter {
    long long v;
    char pad[64 - sizeof(long long)];
};

int main() {
    int T = omp_get_max_threads();

    // Varianta fara padding
    vector<long long> counters(T, 0);

    double start1 = omp_get_wtime();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (long long i = 0; i < ITERS; i++) {
            counters[tid]++;
        }
    }

    double end1 = omp_get_wtime();

    long long total1 = 0;
    for (int i = 0; i < T; i++) total1 += counters[i];

    // Varianta cu padding
    vector<PaddedCounter> padded(T);
    for (int i = 0; i < T; i++) padded[i].v = 0;

    double start2 = omp_get_wtime();

    #pragma omp parallel
```

```
    {
        int tid = omp_get_thread_num();
        for (long long i = 0; i < ITERS; i++) {
            padded[tid].v++;
        }
    }

    double end2 = omp_get_wtime();

    long long total2 = 0;
    for (int i = 0; i < T; i++) total2 += padded[i].v;

    cout << "Fara padding: " << (end1 - start1) << " sec, total = " << total1 << "\n";
    cout << "Cu padding:   " << (end2 - start2) << " sec, total = " << total2 << "\n";

    return 0;
}
```

```
vboxuser@Ubuntu:~/Desktop$ nano activitatea4.c
vboxuser@Ubuntu:~/Desktop$ g++ activitatea4.c -O2 -fopenmp -o activitatea4
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=4
vboxuser@Ubuntu:~/Desktop$ ./activitatea4
Fara padding: 0.0360891 sec, total = 800000000
Cu padding:   0.00747948 sec, total = 800000000
vboxuser@Ubuntu:~/Desktop$
```

CONCLUZIE:

Varianta fără padding este mai lentă din cauza fenomenului de *false sharing*, unde thread-urile accesează aceeași linie de cache. Introducerea padding-ului reduce această problemă și îmbunătățește semnificativ performanța.

ACTIVITATEA 5:

```
#include <omp.h>
#include <iostream>
#include <vector>
#include <cmath>

using namespace std;

static const int N = 20000;

double work(int i) {
    int reps = (i % 1000) + 500; // lucru variabil
    double x = 0.0;
    for (int k = 0; k < reps; k++) {
        x += sin(i * 0.001) * cos(k * 0.002);
    }
    return x;
}

int main() {
    vector<double> out1(N), out2(N);

    // Varianta A - omp for dynamic
    double start1 = omp_get_wtime();

    #pragma omp parallel for schedule(dynamic, 8)
    for (int i = 0; i < N; i++) {
        out1[i] = work(i);
    }

    double end1 = omp_get_wtime();

    // Varianta A2 - omp for guided
    double start2 = omp_get_wtime();

    #pragma omp parallel for schedule(guided, 8)
    for (int i = 0; i < N; i++) {
        out1[i] = work(i);
    }

    double end2 = omp_get_wtime();

    // Varianta B - taskloop
```

```
    double start3 = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        {
            #pragma omp taskloop grainsize(8)
            for (int i = 0; i < N; i++) {
                out2[i] = work(i);
            }
        }
    }

    double end3 = omp_get_wtime();

    cout << "omp for dynamic, chunk=8: " << (end1 - start1) << " sec\n";
    cout << "omp for guided,  chunk=8: " << (end2 - start2) << " sec\n";
    cout << "taskloop, grainsize=8:    " << (end3 - start3) << " sec\n";

    return 0;
}
```

```
vboxuser@Ubuntu:~/Desktop$ g++ activitatea5.c -O2 -fopenmp -o activitatea5
vboxuser@Ubuntu:~/Desktop$ ./activitatea5
omp for dynamic, chunk=32: 0.172007 sec
omp for guided,  chunk=32: 0.0872612 sec
taskloop, grainsize=32:    0.332914 sec
vboxuser@Ubuntu:~/Desktop$ nano activitatea5.c
vboxuser@Ubuntu:~/Desktop$ g++ activitatea5.c -O2 -fopenmp -o activitatea5
vboxuser@Ubuntu:~/Desktop$ ./activitatea5
omp for dynamic, chunk=16: 0.166197 sec
omp for guided,  chunk=16: 0.0848733 sec
taskloop, grainsize=16:    0.384782 sec
vboxuser@Ubuntu:~/Desktop$ nano activitatea5.c
vboxuser@Ubuntu:~/Desktop$ g++ activitatea5.c -O2 -fopenmp -o activitatea5


vboxuser@Ubuntu:~/Desktop$
vboxuser@Ubuntu:~/Desktop$
vboxuser@Ubuntu:~/Desktop$ ./activitatea5
omp for dynamic, chunk=8: 0.170283 sec
omp for guided,  chunk=8: 0.110361 sec
taskloop, grainsize=8:    0.36153 sec
vboxuser@Ubuntu:~/Desktop$
```

Metodă	**Chunk/Grainsize**	**Timp (s)**
omp for dynamic	8	0.170283
omp for dynamic	16	0.166197
omp for dynamic	32	0.172007
omp for guided	8	0.110361
omp for guided	16	0.0848733
omp for guided	32	0.0872612
taskloop	8	0.36153
taskloop	16	0.384782
taskloop	32	0.332914

CONCLUZIE:

Cea mai bună performanță a fost obținută cu omp for guided, în special pentru chunk=16. omp for dynamic are performanță bună, dar mai slabă decât guided, iar taskloop este cea mai lentă variantă din cauza overhead-ului de creare a task-urilor.